

MySQL — Complete Study Notes

From Setup to Advanced Concepts

Prepared by: Anand Soni

1. Introduction to MySQL

1.1 What is MySQL?

MySQL is one of the most widely used open-source relational database management systems (RDBMS) in the world. It organises data into structured tables and lets you interact with that data using SQL (Structured Query Language).

Key highlights:

- Developed by Oracle Corporation
- Uses SQL for querying and managing data
- Supports large-scale web applications (used by Facebook, Twitter, YouTube)
- Available for Windows, macOS, and Linux

1.2 What is a DBMS?

A Database Management System (DBMS) is software that acts as an interface between the user and the database. It handles storing, retrieving, and managing data efficiently. MySQL is a specific type of DBMS called an RDBMS — the "R" stands for relational, meaning data is stored in related tables.

1.3 MySQL Workbench

MySQL Workbench is a visual tool for interacting with MySQL. Instead of typing every command in the terminal, Workbench gives you a graphical interface to design databases, run SQL queries, manage users, and configure the server.

2. Installation Guide

2.1 Windows / macOS

1. Go to <https://dev.mysql.com/downloads/installer/>

2. Download and launch the installer
3. Select "Developer Default" when choosing setup type
4. Set a strong root password during configuration
5. Optionally install MySQL Workbench for a GUI

2.2 Linux (Ubuntu / Debian)

Step 1 — Update the package index:

```
sudo apt update
```

Step 2 — Install MySQL Server:

```
sudo apt install mysql-server
```

Step 3 — Run the security wizard:

```
sudo mysql_secure_installation
```

Step 4 — Create a dedicated user (replace anand and yourpassword):

```
sudo mysql
CREATE USER 'anand'@'localhost' IDENTIFIED BY 'yourpassword';
GRANT ALL PRIVILEGES ON *.* TO 'anand'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;
EXIT;
```

Step 5 — Test your login:

```
mysql -u anand -p
```

3. Databases and Tables

3.1 Understanding the Hierarchy

Think of a MySQL database like a filing cabinet. The cabinet itself is the database, each drawer is a table, and every sheet of paper inside a drawer is a row of data.

Level	Analogy	MySQL Term
Top level	Filing cabinet	Database
Inside cabinet	Drawer / folder	Table
Inside drawer	Single document	Row (Record)
Document field	Column on that doc	Column (Field)

3.2 Creating and Selecting a Database

```
-- Create a new database
CREATE DATABASE school_db;

-- Switch to it
USE school_db;
```

3.3 Creating a Table

Let's create a students table to track student records:

```
CREATE TABLE students (
    student_id    INT AUTO_INCREMENT PRIMARY KEY,
    full_name     VARCHAR(120) NOT NULL,
    email         VARCHAR(120) UNIQUE NOT NULL,
    branch        ENUM('CSE', 'ECE', 'ME', 'CE', 'IT') DEFAULT 'CSE',
    dob           DATE,
    fees_paid     DECIMAL(10, 2) DEFAULT 0.00,
    enrolled_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3.4 Dropping a Database or Table

```
-- Remove the entire database (irreversible!)
DROP DATABASE school_db;

-- Remove just one table
DROP TABLE students;
```

4. Data Types and Constraints

4.1 Common Data Types

Data Type	Description	Example Use
INT	Whole number	student_id, age
VARCHAR(n)	Variable-length text (max n chars)	name, email
DECIMAL(p,s)	Exact decimal with p digits, s after point	fees, salary
DATE	Date value (YYYY-MM-DD)	dob, admission_date
TIMESTAMP	Date + Time, auto-set on insert	created_at

Data Type	Description	Example Use
BOOLEAN	TRUE or FALSE	is_active, is_paid
ENUM(...)	One value from a predefined list	branch, gender
TEXT	Long-form text (no length limit)	description, notes

4.2 Common Constraints

Constraint	Purpose
PRIMARY KEY	Uniquely identifies every row; cannot be NULL
AUTO_INCREMENT	Automatically assigns next integer when a row is inserted
NOT NULL	Prevents a column from having an empty/null value
UNIQUE	All values in this column must be distinct
DEFAULT	Fills in a value automatically if none is provided
CHECK	Validates that a value meets a specified condition
FOREIGN KEY	Links a column to a primary key in another table

5. Modifying Tables with ALTER

5.1 Add a Column

```
ALTER TABLE students ADD COLUMN phone VARCHAR(15);
```

5.2 Remove a Column

```
ALTER TABLE students DROP COLUMN phone;
```

5.3 Change Data Type

```
ALTER TABLE students MODIFY COLUMN full_name VARCHAR(200);
```

5.4 Rename a Column

```
-- CHANGE renames and can also change data type
ALTER TABLE students CHANGE COLUMN email student_email VARCHAR(120);
```

5.5 Reorder Columns

```
-- Move a column to the first position
ALTER TABLE students MODIFY COLUMN email VARCHAR(120) FIRST;

-- Place a column after another
ALTER TABLE students MODIFY COLUMN branch ENUM('CSE','ECE','ME','CE','IT')
AFTER full_name;
```

5.6 Rename a Table

```
RENAME TABLE students TO learners;
```

6. Inserting Data (INSERT INTO)

6.1 Insert a Single Row — Named Columns (Recommended)

```
INSERT INTO students (full_name, email, branch, dob, fees_paid) VALUES
('Anand Soni', 'anand@example.com', 'CSE', '2002-04-10', 45000.00);
```

6.2 Insert Multiple Rows at Once

```
INSERT INTO students (full_name, email, branch, dob, fees_paid) VALUES
('Rahul Verma', 'rahul@example.com', 'ECE', '2001-08-22', 42000.00),
('Priya Sharma', 'priya@example.com', 'IT', '2003-01-15', 47000.00),
('Mohit Jain', 'mohit@example.com', 'ME', '2000-11-30', 38000.00);
```

6.3 Insert Without Specifying Columns

You can omit column names only if you provide values for every column in the exact order they appear in the table. This is fragile — avoid it when possible.

```
INSERT INTO students VALUES
(NULL, 'Test User', 'test@example.com', 'CE', '1999-07-07', 50000.00,
DEFAULT);
```

7. Querying Data (SELECT)

7.1 Basic SELECT

```
-- Fetch every column and row
SELECT * FROM students;

-- Fetch specific columns only
```

```
SELECT full_name, email, branch FROM students;
```

7.2 Filtering with WHERE

```
-- Exact match
```

```
SELECT * FROM students WHERE branch = 'CSE';
```

```
-- Not equal
```

```
SELECT * FROM students WHERE branch != 'ME';
```

```
-- Range
```

```
SELECT * FROM students WHERE fees_paid BETWEEN 40000 AND 50000;
```

```
-- Multiple values
```

```
SELECT * FROM students WHERE branch IN ('CSE', 'IT', 'ECE');
```

```
-- Pattern matching
```

```
SELECT * FROM students WHERE full_name LIKE 'A%'; -- starts with A
```

```
SELECT * FROM students WHERE email LIKE '%@gmail%'; -- contains @gmail
```

```
-- NULL checks
```

```
SELECT * FROM students WHERE dob IS NULL;
```

```
SELECT * FROM students WHERE dob IS NOT NULL;
```

7.3 Logical Operators

```
SELECT * FROM students WHERE branch = 'CSE' AND fees_paid > 40000;
```

```
SELECT * FROM students WHERE branch = 'ECE' OR branch = 'IT';
```

```
SELECT * FROM students WHERE NOT branch = 'ME';
```

7.4 Sorting and Limiting Results

```
-- Sort ascending (default)
```

```
SELECT * FROM students ORDER BY full_name ASC;
```

```
-- Sort descending
```

```
SELECT * FROM students ORDER BY fees_paid DESC;
```

```
-- Top 5 rows
```

```
SELECT * FROM students LIMIT 5;
```

```
-- Pagination: skip first 10, get next 5
```

```
SELECT * FROM students ORDER BY student_id LIMIT 5 OFFSET 10;
```

7.5 DISTINCT — Remove Duplicates

```
SELECT DISTINCT branch FROM students;
```

8. Updating and Deleting Data

8.1 UPDATE

```
-- Update one column
UPDATE students SET fees_paid = 50000.00 WHERE student_id = 1;

-- Update multiple columns
UPDATE students
SET full_name = 'Anand K. Soni', email = 'anand.k@example.com'
WHERE student_id = 1;

-- Arithmetic update (add to existing value)
UPDATE students SET fees_paid = fees_paid + 5000 WHERE branch = 'CSE';
```

⚠ Warning: Omitting WHERE in UPDATE will affect every row in the table.

8.2 DELETE

```
-- Remove one specific row
DELETE FROM students WHERE student_id = 4;

-- Remove rows matching a condition
DELETE FROM students WHERE fees_paid < 30000;

-- Remove all rows (table structure stays)
DELETE FROM students;

-- Faster alternative to clear all rows
TRUNCATE TABLE students;
```

Best practice: Run a SELECT with the same WHERE clause first to preview affected rows before deleting.

9. SQL Functions

9.1 Aggregate Functions

```

SELECT COUNT(*)                AS total_students  FROM students;
SELECT COUNT(*) FROM students WHERE branch = 'CSE';
SELECT SUM(fees_paid)          AS total_fees      FROM students;
SELECT AVG(fees_paid)          AS avg_fees        FROM students;
SELECT MIN(fees_paid), MAX(fees_paid)             FROM students;

-- Group by branch
SELECT branch, AVG(fees_paid) AS avg_fees
FROM students
GROUP BY branch;

```

9.2 String Functions

```

SELECT full_name, LENGTH(full_name)          AS name_len   FROM students;
SELECT full_name, UPPER(full_name)           AS upper_name FROM students;
SELECT full_name, LOWER(full_name)           AS lower_name FROM students;
SELECT CONCAT(full_name, ' <', email, '>') AS contact     FROM students;

```

9.3 Date Functions

```

SELECT NOW();                                -- Current date and time
SELECT CURDATE();                            -- Current date only

SELECT full_name, YEAR(dob)  AS birth_year FROM students;
SELECT full_name, MONTH(dob) AS birth_mon  FROM students;

-- Age in years
SELECT full_name,
       TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age
FROM students;

```

9.4 Conditional: IF()

```

SELECT full_name,
       fees_paid,
       IF(fees_paid >= 45000, 'Full Fee', 'Partial Fee') AS payment_status
FROM students;

```

10. GROUP BY and HAVING

10.1 GROUP BY

GROUP BY collapses rows that share the same value in a column into a single summary row. It is almost always paired with an aggregate function.

```
-- Total fees collected per branch
SELECT branch, SUM(fees_paid) AS total_fees
FROM students
GROUP BY branch;
```

```
-- Student count per branch
SELECT branch, COUNT(*) AS total
FROM students
GROUP BY branch;
```

10.2 HAVING — Filter After Grouping

WHERE filters individual rows before grouping. HAVING filters the resulting groups after aggregation.

```
-- Only show branches where average fees exceed 44000
SELECT branch, AVG(fees_paid) AS avg_fees
FROM students
GROUP BY branch
HAVING AVG(fees_paid) > 44000;
```

```
-- Branches with more than 2 students enrolled
SELECT branch, COUNT(*) AS total
FROM students
GROUP BY branch
HAVING COUNT(*) > 2;
```

Clause	Works On	Supports Aggregates?
WHERE	Individual rows (before grouping)	No
GROUP BY	Organises rows into groups	N/A
HAVING	Groups (after aggregation)	Yes

11. Joining Tables

11.1 Setup — Creating a courses Table

```
CREATE TABLE courses (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    course_name VARCHAR(100),
```

```

        grade          CHAR(2),
        FOREIGN KEY (student_id) REFERENCES students(student_id)
    );

INSERT INTO courses (student_id, course_name, grade) VALUES
(1, 'Database Systems', 'A'),
(2, 'Operating Systems', 'B+'),
(5, 'Computer Networks', 'A-');  -- student_id 5 may not exist

```

11.2 INNER JOIN — Only Matching Rows

```

SELECT students.full_name, courses.course_name, courses.grade
FROM students
INNER JOIN courses ON students.student_id = courses.student_id;

```

Result: Only students who are enrolled in at least one course appear.

11.3 LEFT JOIN — All Left, Matching Right

```

SELECT students.full_name, courses.course_name
FROM students
LEFT JOIN courses ON students.student_id = courses.student_id;

```

Result: Every student appears. Students with no courses show NULL in course_name.

11.4 RIGHT JOIN — All Right, Matching Left

```

SELECT students.full_name, courses.course_name
FROM students
RIGHT JOIN courses ON students.student_id = courses.student_id;

```

Result: Every course appears. Courses with no valid student show NULL in full_name.

JOIN Type	What it Returns
INNER JOIN	Only rows that match in both tables
LEFT JOIN	All rows from left table + matched rows from right (NULL if no match)
RIGHT JOIN	All rows from right table + matched rows from left (NULL if no match)

12. Subqueries

A subquery is a SELECT statement nested inside another query. The inner query runs first and its result is passed to the outer query.

12.1 Scalar Subquery (returns one value)

```
-- Students who paid more than the average
SELECT full_name, fees_paid
FROM students
WHERE fees_paid > (SELECT AVG(fees_paid) FROM students);
```

12.2 Subquery with IN (returns a list)

```
-- Students who are enrolled in at least one course
SELECT full_name
FROM students
WHERE student_id IN (SELECT student_id FROM courses);
```

12.3 Subquery in SELECT Column

```
-- Show each student's fee alongside the overall average
SELECT full_name,
       fees_paid,
       (SELECT AVG(fees_paid) FROM students) AS overall_avg
FROM students;
```

13. Views

A view is a saved SELECT query stored under a name. It behaves like a virtual table — you can query it just like a real table, but it always reflects the latest underlying data.

13.1 Create a View

```
-- View: CSE students who paid full fees
CREATE VIEW cse_full_fee AS
SELECT student_id, full_name, fees_paid
FROM students
WHERE branch = 'CSE' AND fees_paid >= 45000;
```

13.2 Query and Drop a View

```
SELECT * FROM cse_full_fee;

DROP VIEW cse_full_fee;
```

Views are ideal for hiding complexity, restricting column access, or creating reusable query templates.

14. Indexes

An index speeds up data retrieval by creating an internal lookup structure — similar to the index at the back of a textbook. Without an index, MySQL must scan every row (full table scan).

14.1 View, Create, and Drop Indexes

```
-- See existing indexes
SHOW INDEXES FROM students;

-- Index on a single column
CREATE INDEX idx_email ON students(email);

-- Composite index (multi-column)
CREATE INDEX idx_branch_fees ON students(branch, fees_paid);

-- Remove an index
DROP INDEX idx_email ON students;
```

Scenario	Use Index?
Column used frequently in WHERE or JOIN	Yes
Column used in ORDER BY on large tables	Yes
Column that is rarely queried	No — wastes space
Very small table (< 1000 rows)	No — full scan is fast enough

15. Transactions

A transaction is a group of SQL statements that are treated as a single unit. Either all of them succeed (COMMIT) or all are rolled back (ROLLBACK). This protects data integrity.

15.1 AutoCommit

By default MySQL commits every statement automatically. Turn this off for manual control:

```
SET autocommit = 0;  -- disable auto-save
```

```

UPDATE students SET fees_paid = 55000 WHERE student_id = 1;
UPDATE students SET fees_paid = 55000 WHERE student_id = 2;

COMMIT;           -- save both changes
-- OR
ROLLBACK;         -- undo both changes

SET autocommit = 1; -- re-enable auto-save

```

16. Foreign Keys

A foreign key enforces a relationship between two tables. The child table's foreign key column must contain a value that exists in the parent table's primary key column.

16.1 Create with Foreign Key

```

CREATE TABLE assignments (
    assignment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id    INT NOT NULL,
    title         VARCHAR(200),
    submitted_on  DATE,
    CONSTRAINT fk_student
        FOREIGN KEY (student_id) REFERENCES students(student_id)
        ON DELETE CASCADE
);

```

16.2 ON DELETE Options

Option	What Happens When Parent Row is Deleted
RESTRICT (default)	Error — deletion blocked if child rows exist
CASCADE	Child rows are automatically deleted
SET NULL	Foreign key in child row becomes NULL

16.3 Add / Drop Foreign Key with ALTER

```

-- Add after creation
ALTER TABLE assignments
ADD CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES
students(student_id);

```

```
-- Remove it
ALTER TABLE assignments DROP FOREIGN KEY fk_student;
```

17. UNION and UNION ALL

UNION combines the result sets of two SELECT statements into one. Both queries must return the same number of columns with compatible data types.

```
-- Combine names from two tables, remove duplicates
SELECT full_name FROM students
UNION
SELECT full_name FROM alumni;

-- Keep all rows including duplicates
SELECT full_name FROM students
UNION ALL
SELECT full_name FROM alumni;

-- Add a label to distinguish source
SELECT full_name, 'Student' AS type FROM students
UNION
SELECT full_name, 'Alumni' AS type FROM alumni
ORDER BY full_name;
```

18. Stored Procedures

A stored procedure is a named, reusable block of SQL. Define it once, call it whenever needed.

18.1 Creating a Procedure

```
DELIMITER $$
CREATE PROCEDURE EnrollStudent(
    IN p_name    VARCHAR(120),
    IN p_email   VARCHAR(120),
    IN p_branch  ENUM('CSE', 'ECE', 'ME', 'CE', 'IT'),
    IN p_dob     DATE,
    IN p_fees    DECIMAL(10,2)
)
BEGIN
```

```
INSERT INTO students (full_name, email, branch, dob, fees_paid)
VALUES (p_name, p_email, p_branch, p_dob, p_fees);
END$$
DELIMITER ;
```

18.2 Calling and Removing a Procedure

```
CALL EnrollStudent('Riya Gupta', 'riya@example.com', 'IT', '2003-06-18',
46000.00);
```

```
SHOW PROCEDURE STATUS WHERE Db = 'school_db';
```

```
DROP PROCEDURE IF EXISTS EnrollStudent;
```

19. Triggers

A trigger is a stored program that runs automatically in response to an INSERT, UPDATE, or DELETE event on a specified table.

19.1 Create a Log Table

```
CREATE TABLE enrollment_log (
    log_id      INT AUTO_INCREMENT PRIMARY KEY,
    student_id  INT,
    full_name   VARCHAR(120),
    logged_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

19.2 Create a Trigger

```
DELIMITER $$
CREATE TRIGGER trg_after_enroll
AFTER INSERT ON students
FOR EACH ROW
BEGIN
    INSERT INTO enrollment_log (student_id, full_name)
    VALUES (NEW.student_id, NEW.full_name);
END$$
DELIMITER ;
```

NEW.column refers to the newly inserted row's values. Use OLD.column for UPDATE or DELETE.

19.3 Drop a Trigger

```
DROP TRIGGER IF EXISTS trg_after_enroll;
```

20. Quick Reference Cheat Sheet

Command	Purpose
CREATE DATABASE db_name	Create a new database
USE db_name	Switch to a database
CREATE TABLE tbl (...)	Define a new table with columns
ALTER TABLE tbl ADD COLUMN ...	Add a column to an existing table
DROP TABLE tbl	Delete a table permanently
INSERT INTO tbl (...) VALUES (...)	Add one or more rows
SELECT ... FROM tbl WHERE ...	Query and filter rows
UPDATE tbl SET col=val WHERE ...	Modify existing rows
DELETE FROM tbl WHERE ...	Remove specific rows
TRUNCATE TABLE tbl	Clear all rows, keep structure
INNER / LEFT / RIGHT JOIN	Combine rows from related tables
GROUP BY + HAVING	Group rows and filter groups
CREATE VIEW v AS SELECT ...	Save a query as a virtual table
CREATE INDEX idx ON tbl(col)	Speed up lookups on a column
SET autocommit = 0 / COMMIT / ROLLBACK	Manual transaction control
CREATE PROCEDURE / CALL	Define and execute reusable SQL blocks
CREATE TRIGGER	Auto-execute SQL on table events